

Personality Prediction System via CV Analysis

Thanh Hai Nguyen (619562) · Khanh Tran Nguyen (619956)

CS 582 Machine Learning - Group 3

Abstract

We build a system that predicts a candidate's Big Five personality traits (Extraversion, Neuroticism, Agreeableness, Conscientiousness, Openness) from the text of a CV. The resume corpus we use has no personality labels, so we train on a separate labeled text corpus (2,467 essays with self-reported Big Five) and transfer the models to 2,484 real resumes. We compare a baseline that follows Grunenberg et al. (2024) - TF-IDF + SVD features with a Random Forest - against an improved pipeline of Sentence-BERT embeddings, XGBoost and SHAP explanations. Both models reach a 5-fold out-of-fold correlation of $r \approx 0.12$ on average, with Openness the most predictable trait ($r \approx 0.23-0.25$). On resumes, the improved model separates job categories about three times more strongly than the baseline. The models are served through a FastAPI backend with a React front end that accepts a pasted CV or a PDF upload.

1. Introduction

1.1 Personality prediction in recruiting

Recruiters often want to know a candidate's personality, but questionnaires are slow and easy to fake. Grunenberg et al. (2024) showed that machine learning can estimate the Big Five from the text of a CV and short written answers, with predictions that relate to external criteria such as vocational interests. Unlike earlier work that used social-media likes or posts (Youyou et al., 2015; Park et al., 2015), this uses only documents a candidate already hands in, which avoids the privacy and legal problems of scraping social media.

1.2 The Big Five model

The Big Five describes personality on five broad dimensions:

- **Extraversion** - sociable, energetic, talkative
- **Neuroticism** - prone to anxiety and negative emotion

- **Agreeableness** - cooperative, warm, trusting
- **Conscientiousness** - organised, reliable, goal-driven
- **Openness** - curious, creative, open to new ideas

In our training data each trait is a binary label (high / low), so we treat each trait as its own binary classification problem and use the predicted probability as a 0-1 trait score.

1.3 Goals

1. Re-implement the paper's text-based approach as a baseline.
2. Improve it with contextual sentence embeddings, gradient boosting and explainability (SHAP), which the paper lists as future work.
3. Check whether predictions on real resumes vary sensibly with job category, standing in for the paper's vocational-interest validation.
4. Deploy the models in a usable web application.

2. Dataset

2.1 Problem with the dataset

Our target data, the Kaggle Resume Dataset, has no personality labels - only `ID`, `Resume_str`, `Resume_html` and a job `Category`. A supervised personality model cannot be trained on it directly. The original paper faced the same issue and trained on a separate proprietary recruiting dataset.

2.2 Two datasets, two roles

We solve this by training on a public labeled corpus and applying the model to the resumes.

Dataset	Role	Size	Labels
Essays (Pennebaker & King)	Train and cross-validate	2,467 essays, mean 652 words	Binary Big Five (y/n) per trait; each trait is about 50% positive
Kaggle Resume Dataset	Apply model + criterion validity	2,484 resumes, mean 811 words, 24 job categories	Job <code>Category</code> only

The resumes are also provided as PDFs grouped by category (for example 120 for Information Technology, 22 for BPO). These PDFs serve as realistic inputs for the application's PDF-upload path.

3. Data Preparation

3.1 Problem in data preparation

The essays and the resumes are very different kinds of text: essays are free-flowing personal writing, resumes are formal lists of jobs and skills. A model that reads too much into style or topic may not transfer well. We therefore keep cleaning light and use the same preprocessing for both corpora.

3.2 Preparation steps

1. **Label encoding** - the essay labels `y / n` are mapped to 1/0; rows missing a label or text are dropped (`common.py`).
2. **Text cleaning** - newlines and repeated whitespace are collapsed into single spaces. Punctuation is kept, because both TF-IDF n-grams and sentence embedders benefit from sentence boundaries.
3. **PDF extraction** - for uploaded CVs, text is extracted page by page with `pdfplumber` , the same tool used in the original paper.
4. **Minimum length** - inputs with fewer than 20 words are rejected by the app, since very short text gives unreliable predictions (the paper excluded CVs under 50 words).
5. **Evaluation split** - stratified 5-fold cross-validation (shuffled, seed 42), giving out-of-fold predictions for every essay.

4. Models

We train two pipelines. Each pipeline turns text into a feature vector, then fits five independent binary classifiers, one per trait. After cross-validation, each classifier is refit on all 2,467 essays and saved with its feature extractor in a single `joblib` bundle.

4.1 Baseline: TF-IDF + SVD + Random Forest

This mirrors the paper's word-frequency features without needing a GloVe download. TF-IDF captures which words and word pairs a writer uses; SVD compresses them into 100 dense latent topics, similar to the paper's dimensionality reduction.

4.2 Improved: Sentence-BERT + XGBoost + SHAP

- **Sentence-BERT** (`all-MiniLM-L6-v2`) encodes each document as a 384-dimensional contextual embedding. Unlike bag-of-words, it reflects word order and meaning in context.
- **XGBoost** replaces the Random Forest. Gradient boosting usually fits dense embedding features better, and it trains quickly.
- **SHAP** (`TreeExplainer`) ranks which embedding dimensions push each trait up or down, computed on 200 essays. The paper names explainability as missing future work.

4.3 Settings

Setting	Baseline	Improved
Text features	TF-IDF, 1-2-grams, max 5,000 terms, English stop words removed, min_df = 2	Sentence-BERT <code>all-MiniLM-L6-v2</code>
Feature size	100 (TruncatedSVD)	384
Classifier	Random Forest, 300 trees	XGBoost, 300 trees, depth 4, learning rate 0.05, subsample 0.8, colsample 0.8
Explainability	-	SHAP, top 10 dimensions per trait
Validation	Stratified 5-fold, out-of-fold	Stratified 5-fold, out-of-fold

4.4 Evaluation metric

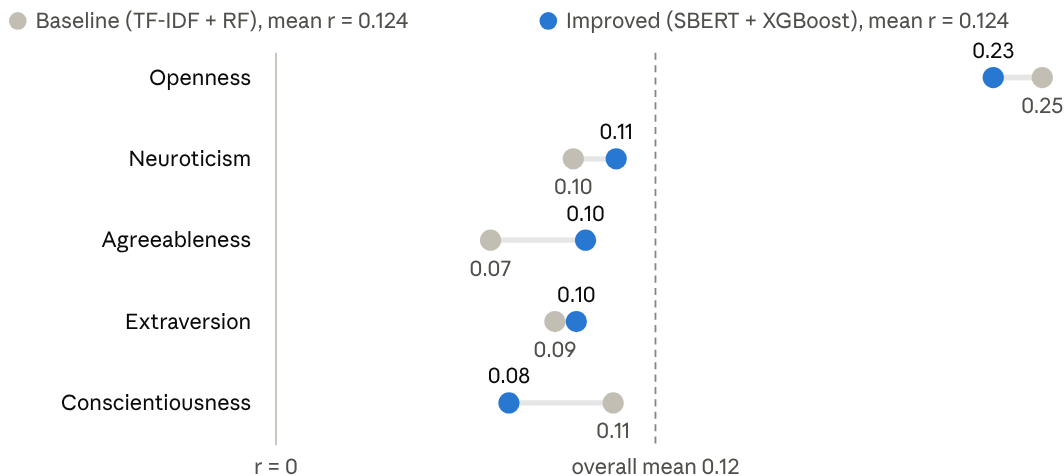
Following the paper, we report the Pearson correlation r between the out-of-fold predicted probability and the self-reported label for each trait. Out-of-fold predictions come from a model that never saw that essay, so r estimates performance on new text.

5. Training and Results

5.1 Personality prediction on essays

Openness is the most predictable trait; both models average $r \approx 0.12$

Out-of-fold Pearson r (predicted probability vs self-report), stratified 5-fold CV on 2,467 essays



`train_baseline.py / train_improved.py · cv_results in models/*.joblib`

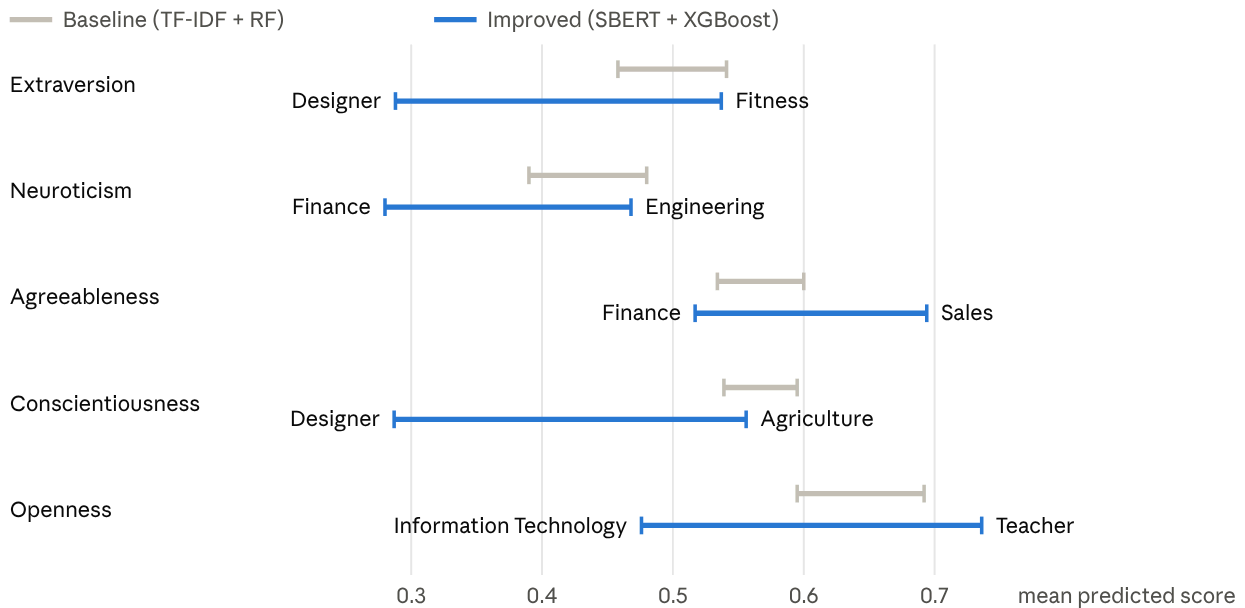
Both models predict four traits only slightly better than chance ($r = 0.07$ – 0.11) and Openness clearly better ($r \approx 0.24$). The improved model gains on Agreeableness and Neuroticism but loses on Conscientiousness, so the averages tie (0.124 vs 0.124). Correlations of this size are useful for group-level patterns, not for judging one person.

5.2 Criterion validity on resumes

The improved model separates job categories about 3× more than the baseline

Range of per-category mean trait score, averaged over traits: 0.23 improved vs 0.08 baseline

300 sampled resumes, 24 job categories; labels name the lowest and highest category (improved model)



```
predict_and_compare.py --sample 300 · predictions_baseline.csv, predictions_improved.csv
```

The baseline gives nearly the same scores to every job category, so it says little about resumes. The improved model spreads categories much more, and several extremes match intuition: Fitness and Teacher score highest on Extraversion and Designer lowest; Teacher and Designer score highest on Openness; Sales scores highest on Agreeableness.

Other results do not fit theory. Information Technology scores lowest on Openness, Designer lowest on Conscientiousness, and Accountant (0.52) is not the most conscientious category. With about 12 resumes per category, these are signals to test on the full dataset, not proven differences.

6. Novelty

Our main additions are contextual embeddings, explainability, a reproducible public training set and a deployed application.

	Grunenberg et al. (2024)	This project
Text features	Averaged GloVe + TF-IDF/SVD + LIWC + LDA (694 features)	TF-IDF/SVD (baseline) or contextual Sentence-BERT (improved)
Model	Random Forest / Lasso	Random Forest (baseline) or XGBoost (improved)
Explainability	None (listed as future work)	SHAP values on the improved model
Training data	Proprietary recruiting data	Public essays corpus, so the work is reproducible
Criterion validity	Self-reported vocational interests	Job <code>Category</code> of real resumes
Deployment	None (research only)	FastAPI REST service + React web app with PDF upload

The transfer setup is itself a contribution: it shows how far a personality model trained on one kind of text carries over to resumes when no labeled resumes exist.

7. Setup

The project has two parts: the model scripts in `source/` and the web application in `app/`.

7.1 The prediction scripts (`source/`)

- `common.py` - shared trait names, data loading and text cleaning
- `train_baseline.py` - trains the TF-IDF + SVD + Random Forest model
- `train_improved.py` - trains the Sentence-BERT + XGBoost model and computes SHAP
- `predict_and_compare.py` - scores `Resume.csv` and prints mean traits per job category
- `extract_cv_text.py` - PDF-to-text with `pdfplumber`
- `app.py` - a simple Streamlit demo with a radar chart

```

pip install -r requirements.txt
python train_baseline.py --data essays.csv --out models/baseline.joblib
python train_improved.py --data essays.csv --out models/improved.joblib
python predict_and_compare.py --model models/improved.joblib --resumes
Resume.csv --out predictions_improved.csv --sample 300

```

7.2 The web application (app/)

The backend is a FastAPI service that loads both model bundles and exposes:

- `POST /api/v1/predict` - JSON `{"text": "...", "model_id": "baseline" | "improved"}`
- `POST /api/v1/extract/pdf` - extract text from an uploaded PDF
- `POST /api/v1/predict/pdf` - extract and score a PDF in one call
- `GET /api/v1/models`, `GET /api/v1/health/ready` - model availability

The frontend (React + TypeScript, Vite) lets the user paste text or upload a PDF, choose a model, and see the five trait scores. It warns when the CV is longer than the Sentence-BERT input limit.

```

# backend
python3.13 -m venv app/.venv
app/.venv/bin/python -m pip install -r app/backend/requirements.txt
cd app && .venv/bin/uvicorn backend.main:app --host 127.0.0.1 --port 8000

# frontend
cd app/frontend && npm install && npm run dev

```

8. Conclusion and Future Work

We built an end-to-end system that predicts Big Five traits from CV text and serves them through a web application. Both models predict personality only weakly (average $r \approx 0.12$), with Openness clearly the easiest trait. Replacing TF-IDF with Sentence-BERT did not raise accuracy on essays, but it produced resume scores that differ much more between job categories, in several cases in theory-consistent directions.

8.1 Limitations

- **Domain shift** - the models learn from student essays, not resumes, so scores on resumes cannot be checked against true personality.

- **Input truncation** - `all-MiniLM-L6-v2` reads at most 256 word pieces, while essays average 652 words and resumes 811, so the improved model sees only the start of each document.
- **Small validation sample** - category means are based on 300 resumes (about 12 per category), so single-category differences are noisy.
- **Interpretability** - SHAP ranks embedding dimensions, which are not human-readable words.
- **Ethics** - scores are probabilities, not a clinical assessment, and should not be used as a final hiring decision tool.

8.2 Future work

- Split long documents into chunks and average their embeddings to avoid truncation.
- Fine-tune a transformer end-to-end, or add LIWC-style features as in the paper.
- Score the full 2,484 resumes and test category differences statistically (for example ANOVA per trait).
- Map SHAP back to words or sentences for explanations a recruiter can read.

References

1. Grunenber, E., Peters, H., Francis, M. J., Back, M. D., & Matz, S. C. (2024). Machine learning in recruiting: predicting personality from CVs and short text responses. *Frontiers in Social Psychology*, 1:1290295. doi.org/10.3389/frsps.2023.1290295
2. Park, G., Schwartz, H. A., Eichstaedt, J. C., et al. (2015). Automatic personality assessment through social media language. *Journal of Personality and Social Psychology*, 108, 934–952.
3. Youyou, W., Kosinski, M., & Stillwell, D. J. (2015). Computer-based personality judgments are more accurate than those made by humans. *PNAS*, 112, 1036–1040.
4. Pennebaker, J. W., & King, L. A. (1999). Linguistic styles: language use as an individual difference. *Journal of Personality and Social Psychology*, 77, 1296–1312.
5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: sentence embeddings using Siamese BERT-networks. *EMNLP 2019*.
6. Resume Dataset, Kaggle. kaggle.com/datasets/snehaanbhawal/resume-dataset